

HerGuardian

Predictive Safety, Safer Routing and Emergency Support

Technical Documentation

Final Project Report

Team Friday

Karan Regmi Kshetri

Samikshya Ranabhat

Niru Thapa

Upesh Bhujel

HerGuardian is a mobile-first Progressive Web Application for location-aware safety assessment, safer-route comparison, emergency SOS support, guardian alerts, evidence recording, and resilient offline emergency functionality.

October 2026

Document Purpose

This document describes the final hackathon implementation of **HerGuardian**. It records the architecture, safety model, routing logic, database and security design, APIs, SOS and guardian workflow, emergency recording, Web Push, offline/PWA behaviour, installation, testing, limitations and future work.

The report intentionally distinguishes between features verified in the current implementation and capabilities that would require a future native or production deployment. In particular, the current safety score is a **transparent deterministic weighted heuristic**, not a trained machine-learning model.

Contents

Document Purpose	i
1 Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Solution Concept	2
1.4 Final Feature Status	2
2 System Architecture	3
2.1 Architecture Overview	3
2.2 Technology Stack	4
2.3 End-to-End Application Flow	4
2.4 Important Final Source Areas	4
3 Safety Scoring and Routing	6
3.1 Safety Model	6
3.2 Scoring Weights	6
3.3 Safety Formula	6
3.4 Time-of-Day Risk	7
3.5 Safety Data and Fallback	7
3.6 Report-Risk Decay	7
3.7 Route Evaluation	7
3.8 Saved Journey Geometry	7
4 Database, Storage and Security Model	8
4.1 Database Platform	8
4.2 Core Tables	8
4.2.1 Safety Cells	8
4.2.2 Incidents	8
4.2.3 Reports	8
4.2.4 Guardian Sessions	8
4.2.5 Push Subscription Sessions	8
4.3 Private Storage	8
4.4 Row-Level Security and Privileged Operations	9
4.5 Guardian Capability Model	9
5 API and External Services	10
5.1 Application APIs	10
5.2 Safety Cells	10
5.3 Scoring	10
5.4 Routes	10
5.5 Reports	10

5.6	SOS	11
5.7	Recording Upload	11
5.8	External Services	11
6	SOS, Voice Activation and Guardian System	12
6.1	Local-First SOS Workflow	12
6.2	Voice SOS	12
6.3	Guardian Alerts and Web Push	13
6.4	Live Guardian Tracking	13
6.5	Calling and SMS	13
6.6	SOS Sound Behaviour	13
7	Emergency Audio Recording	14
7.1	Purpose	14
7.2	Final Recording Strategy	14
7.3	Failure Isolation	14
7.4	Recording Limitations	14
8	Offline and PWA Behaviour	15
8.1	Progressive Web Application	15
8.2	Offline Application Shell	15
8.3	Offline Safety Data	15
8.4	Saved Journeys	15
8.5	Offline Destination Selection	15
8.6	Basemap Tile Caching	15
8.7	Offline SOS and Outbox	16
8.8	Secure Context Requirement	16
9	Installation and Deployment	17
9.1	Requirements	17
9.2	Install Dependencies	17
9.3	Environment Variables	17
9.4	Development	17
9.5	Production Build	17
9.6	Deployment	18
9.7	PWA Installation	18
10	Testing and Validation	19
10.1	Automated Validation	19
10.2	Regression Coverage	19
10.3	Physical Android Verification	19
11	Demonstration Guide	21
11.1	Pre-Demo Checklist	21
11.2	Suggested Demo Flow	21
11.2.1	Step 1: Safety Map	21
11.2.2	Step 2: Transparent Safety Score	21
11.2.3	Step 3: Safer Route Comparison	21
11.2.4	Step 4: Offline Resilience	21
11.2.5	Step 5: SOS	22
11.2.6	Step 6: Guardian Alert and Tracking	22
11.2.7	Step 7: Voice SOS	22

11.2.8 Step 8: Community Feedback Loop	22
12 Security and Privacy	23
12.1 Server-Side Secrets	23
12.2 Private Emergency Media	23
12.3 Anonymous Reports	23
12.4 Guardian Session Privacy	23
12.5 Push Isolation	23
12.6 Location Privacy	23
13 Known Limitations	24
14 Future Improvements	25
15 Conclusion	26

1. Introduction

1.1 Project Overview

HerGuardian is a mobile-first personal-safety system developed by Team Friday. The project attempts to move safety technology beyond a purely reactive panic button by combining prevention, route awareness, emergency response and community feedback in one PWA.

The final prototype combines:

- location-based safety scoring;
- time-sensitive risk evaluation;
- shortest-versus-safest route comparison;
- anonymous community safety reports;
- real browser geolocation;
- one-tap and armed voice-triggered SOS;
- local-first emergency persistence;
- guardian live-location tracking;
- session-specific Web Push notifications;
- private emergency audio recording;
- saved journeys and saved destinations;
- offline safety overlays and previously viewed map tiles; and
- installable PWA behaviour.

1.2 Problem Statement

Many personal-safety applications become useful only after a person already feels threatened. HerGuardian adds a preventive layer by asking:

- How safe is the user's current area according to available factors?
- Does the estimated safety level change with time?
- Is an alternative route safer than the shortest route?
- Can community reports influence later safety evaluations?
- Can an SOS request survive temporary network loss?
- Can a trusted guardian receive an alert and follow the emergency session?

1.3 Solution Concept

The project follows the loop:

Predict → **Prevent** → **Respond** → **Improve**

Safety cells provide the factors used by a transparent scoring function. Candidate routes are obtained from OSRM and evaluated using the same safety model. Community reports increase nearby report risk. During an emergency, HerGuardian creates a persistent SOS session, tracks location, alerts subscribed guardians, and can capture private audio evidence.

1.4 Final Feature Status

Feature	Final behaviour	Status
Safety grid	Location-based cells containing crime, report, lighting and crowd factors.	Implemented
Dynamic safety score	Weighted score using crime, reports, time, lighting and crowd.	Implemented
Safest-route comparison	Scores OSRM alternatives and distinguishes safest from shortest.	Implemented
Anonymous reports	Stores community reports and raises nearby report risk.	Implemented
Real GPS	Browser geolocation drives current position, routing and SOS location.	Implemented
One-tap SOS	Creates or reuses an emergency session with local-first persistence.	Implemented
Voice SOS	Can be armed while the application is active and trigger SOS by voice.	∂
Guardian tracking	Live/reload-safe guardian session view.	Implemented
Web Push	Session-specific guardian subscriptions and emergency notification delivery.	Implemented
Emergency recording	Browser MediaRecorder captures complete private audio segments.	Implemented
Offline SOS	Persists locally and synchronizes when connectivity returns.	Implemented
Offline safety map	Cached safety cells and overlays can render offline.	Implemented
Saved offline journey	Exact previously saved route geometry and endpoints can be reopened offline.	Implemented
Offline basemap	Previously viewed OSM tiles are cached within a bounded cache.	∂
Offline destination selection	Saved places and map-tap coordinates can be selected offline.	Implemented
Full offline routing	New arbitrary road routes still require a routing engine/network.	Future work
Authority analytics	Full production authority dashboard remains outside the final MVP.	Future work

Table 1.1: Final HerGuardian feature status

2. System Architecture

2.1 Architecture Overview

HerGuardian uses a Next.js client/server architecture with Supabase for persistent data, private storage and realtime communication. The browser/PWA handles user interaction, MapLibre rendering, geolocation, IndexedDB, service worker caching, MediaRecorder and Web Push subscription. Privileged database and storage operations remain server-side.

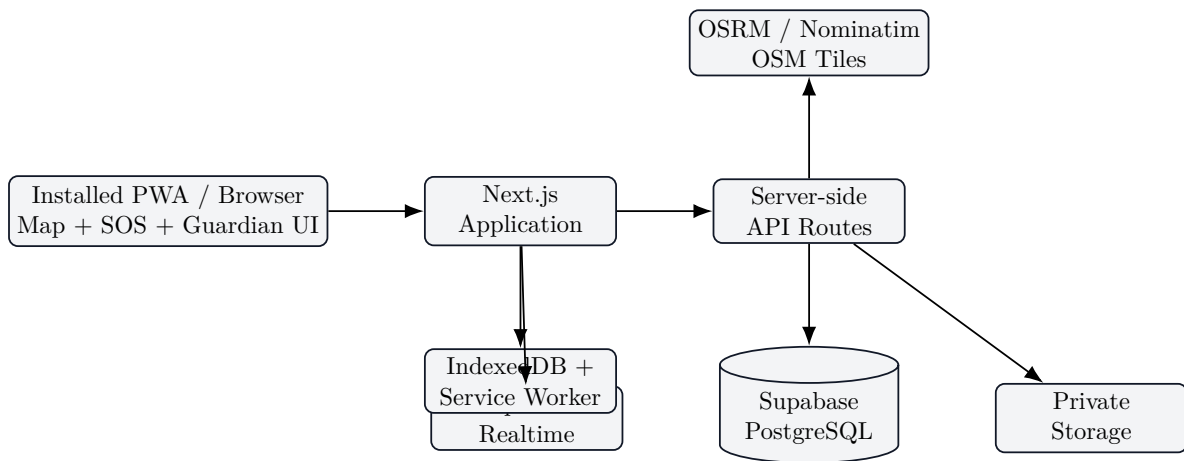


Figure 2.1: Final high-level HerGuardian architecture

2.2 Technology Stack

Technology	Purpose
Next.js 16.3.8	App Router application, server rendering and API routes.
React 19.2.8	Interactive client interface.
TypeScript	Application, safety, offline and emergency logic.
Tailwind CSS 4	Responsive interface styling.
MapLibre GL 6.11.2	Interactive safety, route and guardian maps.
Supabase	PostgreSQL, Realtime and private Storage.
OSRM	Online road-route alternatives.
Nominatim	Online place search/geocoding.
IndexedDB	Offline SOS/outbox, saved data, guardians and queued media.
Service Worker	Offline application shell, workers and bounded map-tile caching.
Web Push	Guardian emergency notifications.
MediaRecorder	Emergency audio capture.
Node.js	Build, test and utility scripts.

Table 2.1: Primary technology stack

2.3 End-to-End Application Flow

1. The PWA requests the user's current geolocation.
2. Safety cells are loaded from the server or validated local cache.
3. The nearest relevant cell is evaluated with the safety formula.
4. The user searches for or selects a destination.
5. Online, OSRM provides road-route alternatives.
6. Route geometry is sampled against safety cells and alternatives are scored.
7. A successful journey stores its endpoints, labels and exact geometry locally.
8. If SOS is activated, emergency state is persisted locally before depending on the network.
9. The server persists/replays the session safely and triggers session-specific guardian notifications when applicable.
10. GPS updates feed the guardian session while emergency audio can be captured privately.
11. On reconnect, queued work is synchronized without intentionally reopening terminal sessions.

2.4 Important Final Source Areas

The final implementation is concentrated in the following areas:

- `src/components/map-view.tsx`: map, route and offline journey UI;
- `src/components/place-search.tsx`: online search and saved-place interaction;
- `src/components/guard-tracker.tsx`: guardian tracking interface;
- `src/lib/safety.ts`: safety-score model;

- `src/lib/map-cache.ts`: saved safety/journey data;
- `src/lib/map-overlays.ts`: map overlay restoration;
- `src/lib/places-cache.ts`: saved destination cache;
- `src/lib/offline.ts`: local outbox and queued binary data;
- `src/lib/recording.ts`: browser recording lifecycle;
- `src/app/api/recording/route.ts`: private recording upload;
- `src/app/(bare)/sos/page.tsx`: emergency UI and recording states;
- `public/sw.js`: production offline/service-worker behaviour.

3. Safety Scoring and Routing

3.1 Safety Model

The safety score is a deterministic weighted heuristic. It is intentionally transparent and reproducible; it should not be presented as a trained AI/ML model.

Each safety cell supplies:

- **crimeRisk**: historical or seeded crime risk;
- **reportRisk**: community-report-derived risk;
- **lighting**: lighting quality, with larger values representing better lighting;
- **crowd**: activity/crowd level, with larger values representing busier areas.

Time risk is calculated dynamically.

3.2 Scoring Weights

Factor	Weight
Crime risk	40%
Community report risk	25%
Time of day	15%
Lighting	10%
Crowd	10%
Total	100%

Table 3.1: Safety-score weights

3.3 Safety Formula

For crime risk C , report risk R , time risk T , lighting L and crowd score D :

$$S = 0.40(1 - C) + 0.25(1 - R) + 0.15(1 - T) + 0.10L + 0.10D$$

The displayed score is:

$$\text{Safety Score} = \text{round}(100S)$$

A larger result represents a safer estimated location under the current model.

3.4 Time-of-Day Risk

Time	Risk
06:00–17:59	0.1
18:00–21:59	0.4
22:00–03:59	1.0
04:00–05:59	0.7

Table 3.2: Time-risk values

3.5 Safety Data and Fallback

The application prefers Supabase-backed safety cells. A deterministic fallback exists so that a temporary backend problem does not make the map completely unusable. Synthetic/fallback data must remain clearly distinguished from validated real-world public-safety data.

The API can expose whether the source is database-backed or fallback data:

```
"source": "db"
```

or

```
"source": "fake"
```

3.6 Report-Risk Decay

Community report risk decays at read time:

$$R_{\text{effective}} = R_{\text{stored}} e^{-d/7}$$

where d is the number of days since the risk value was last bumped. This makes recent reports more influential than older ones.

3.7 Route Evaluation

Online routing uses OSRM [6]. For each returned route:

1. GeoJSON route geometry is obtained.
2. Points are sampled along the geometry.
3. Each sample is associated with nearby safety-cell information.
4. The safety score is calculated at the current time.
5. Sample scores are averaged into a route score.

The application separately identifies the route with the lowest travel time and the route with the highest safety score. Therefore, the safest route does not have to be the shortest route.

3.8 Saved Journey Geometry

A successfully generated route stores the original start, destination, labels and route geometry. When offline, HerGuardian can reopen that exact historical journey without recalculating it or substituting a later GPS position. A new arbitrary road route still requires online routing because the PWA does not bundle a complete offline road graph.

4. Database, Storage and Security Model

4.1 Database Platform

HerGuardian uses Supabase PostgreSQL [2]. Core application data includes safety cells, incidents, reports, guardian sessions and the push subscription/session association required for session-specific notifications. The project also uses private storage for recordings and report media.

4.2 Core Tables

4.2.1 Safety Cells

The **safety_cells** table contains latitude, longitude, crime risk, report risk, report-risk timestamp, lighting, crowd and update metadata.

4.2.2 Incidents

The **incidents** table contains seeded/historical incident information such as category, severity, location, occurrence time and source/provenance.

4.2.3 Reports

The **reports** table stores anonymous community safety reports. Raw reports are not intended to be publicly enumerable.

4.2.4 Guardian Sessions

The **guardian_sessions** table stores the emergency lifecycle, trigger source, latest location, media paths and timing information. The session UUID also participates in the guardian capability model.

4.2.5 Push Subscription Sessions

The final notification design associates push subscriptions with the specific SOS/guardian session rather than broadcasting every emergency to every stored subscription. This provides recipient isolation at the application level.

4.3 Private Storage

The project uses private Supabase Storage buckets including:

```
recordings  
report-photos
```

The application stores object paths rather than permanent public recording URLs. Emergency recordings remain private and privileged upload operations are performed server-side.

4.4 Row-Level Security and Privileged Operations

Row-Level Security is enabled on relevant application tables. Public access is kept narrow: safety data can be read where required, anonymous reports can be submitted, while privileged guardian-session mutation and emergency persistence are handled through server-side logic/RPCs using the service role. Raw reports and private recordings are not exposed as public datasets.

The final emergency persistence path includes replay-safe server-side operations for initial and terminal SOS state, and push subscription association. The service-role key must never be sent to browser code.

4.5 Guardian Capability Model

The hackathon MVP uses the guardian session identifier as a capability in the shared guardian link. Anyone possessing a valid link should be treated as having access to that session. This is simpler than a production guardian account system and is documented as a limitation rather than strong identity proof.

5. API and External Services

5.1 Application APIs

The Next.js server layer provides endpoints for the main application flows. Important endpoint families include:

- safety-cell retrieval and location scoring;
- route retrieval and route-safety evaluation;
- anonymous report submission;
- SOS creation, replay, location and terminal-state updates;
- guardian session retrieval;
- push subscription/notification support; and
- private emergency recording upload.

5.2 Safety Cells

GET `/api/cells` returns safety cells and a source indicator. The client uses validated non-empty responses and can preserve the last valid dataset for offline use.

5.3 Scoring

The scoring API accepts a location and returns the calculated safety score and its underlying factors. The same safety logic is reused for route evaluation.

5.4 Routes

The route endpoint validates start/destination coordinates, requests online road alternatives from OSRM, scores those alternatives, and returns route geometry plus distance, duration and safety information.

5.5 Reports

Anonymous reports are validated server-side. A successful report can increase report risk in nearby safety cells while keeping raw report records from public read access.

5.6 SOS

The client creates an emergency identifier locally before relying on network availability. Server-side persistence is designed to be replay-safe so queued requests can be retried without intentionally creating duplicate active sessions or reopening terminal sessions.

5.7 Recording Upload

The final recording API validates non-empty binary uploads, preserves MIME metadata, uses stable retry-safe naming, uploads to the private recordings bucket, and appends recording references to the relevant emergency session.

5.8 External Services

- **OSRM**: online road routing [6].
- **Nominatim**: online place search/geocoding; the project does not claim full offline geocoding [7].
- **OpenStreetMap tiles**: visual basemap, subject to bounded normal-use caching and OSM tile policy [8].

6. SOS, Voice Activation and Guardian System

6.1 Local-First SOS Workflow

The emergency path prioritizes persistence of the SOS itself over secondary features such as audio upload.

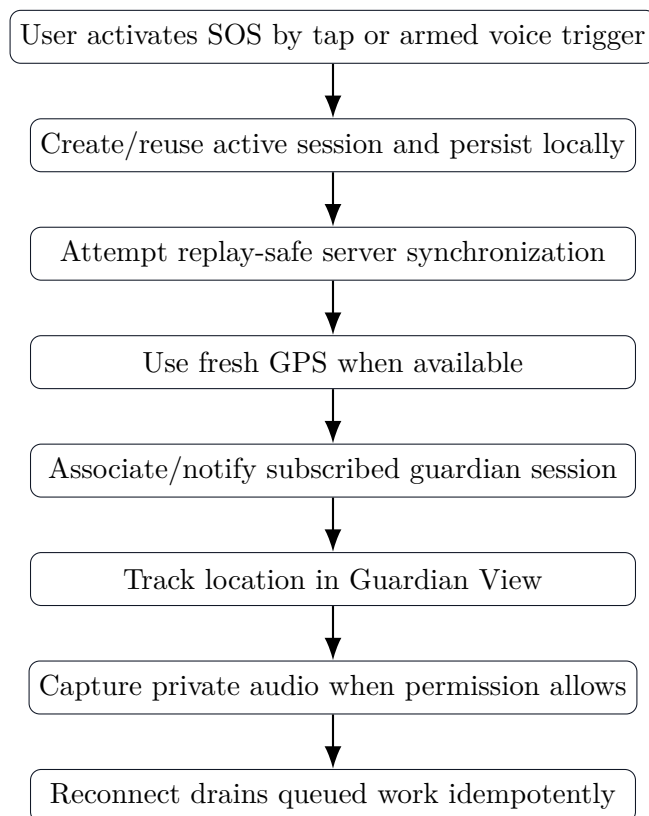


Figure 6.1: Final emergency workflow

6.2 Voice SOS

Voice SOS is a preventive, armed feature. Before a risky journey, the user can arm voice detection while the application remains active. A recognized help phrase can then activate the same SOS workflow without requiring the user to tap the screen.

This is **not** presented as always-on locked-screen detection. Browser and mobile operating-system lifecycle restrictions can suspend a PWA when it is closed, backgrounded or the device is locked. Always-on locked-screen emergency voice activation is therefore future native-app work.

6.3 Guardian Alerts and Web Push

Web Push follows the standard browser push/service-worker model [12]. Guardian subscriptions are associated with the relevant session so an alert is not intentionally broadcast to unrelated guardians. Delivery still depends on browser/OS push infrastructure and network availability and therefore should not be described as guaranteed.

6.4 Live Guardian Tracking

The guardian view combines live updates with persisted state. This provides a reload/fallback path if an immediate realtime update is missed. The guardian map centers on the first available position and continues to update as the session changes.

6.5 Calling and SMS

The PWA can provide explicit **Call guardian** and **Send location by SMS/share** actions. It does not silently place a cellular call or silently send an SMS through the user's SIM. Such actions require the phone/browser to hand off to the relevant user-facing application or share flow.

6.6 SOS Sound Behaviour

The final stabilization removed the automatic Web Audio siren/oscillator from normal SOS activation. Emergency persistence, guardian notification and recording are independent of that sound behaviour.

7. Emergency Audio Recording

7.1 Purpose

When microphone permission is available, HerGuardian can capture emergency audio evidence associated with an SOS session. The feature uses the browser MediaRecorder API [11] and stores completed recordings in the private Supabase recordings bucket.

7.2 Final Recording Strategy

An earlier implementation emitted MediaRecorder data on a short timeslice and uploaded individual fragments as separate WebM objects. Arbitrary timeslice fragments are not a reliable guarantee of independently playable media files. The final implementation instead creates completed recording segments, normally around 30 seconds, and combines the binary parts of each recorder session before committing/uploading the segment.

The final behaviour includes:

- browser-supported MIME selection using capability checks;
- preference for common audio formats such as WebM/Opus where supported;
- rejection of empty recordings;
- prevention of overlapping recorder sessions;
- cleanup of microphone tracks after recording;
- preservation of binary data and MIME metadata through offline queuing;
- stable retry-safe upload naming;
- private storage; and
- truthful UI states for recording, queued/uploading, saved and failed conditions.

7.3 Failure Isolation

Microphone permission denial, recording failure or upload failure must not cancel the SOS itself. SOS persistence is higher priority than evidence capture. Completed offline audio can remain queued for synchronization after connectivity returns.

7.4 Recording Limitations

Browser background execution is not guaranteed. Abrupt operating-system termination can lose an unfinished segment. Media container duration metadata can also be interpreted differently by preview tools; real-device playback is therefore part of the final physical-device verification process.

8. Offline and PWA Behaviour

8.1 Progressive Web Application

HerGuardian is installable as a PWA. The service worker is registered in production, while development mode deliberately avoids retaining a stale production worker on localhost. Service workers provide offline/network interception capabilities in supported secure contexts [9].

8.2 Offline Application Shell

Map, Guardians, Rights, Help, Login and SOS have distinct cached application pages/assets so offline navigation does not incorrectly return a single map page for unrelated routes.

8.3 Offline Safety Data

A valid non-empty safety dataset is retained locally. Failed, empty or fallback responses are prevented from silently overwriting a known-good saved dataset. MapLibre safety sources/layers are restored after style changes. This behaviour was verified on the physical Android test device with safety circles visible after connectivity was disabled.

8.4 Saved Journeys

Successful online journeys store their original endpoints, labels and exact route geometry. The displayed route is not intentionally erased merely because connectivity disappears. After a cold offline reopen, the user can select **Reopen last saved journey** to restore the historical route without an OSRM request. Current GPS drift does not replace the saved historical start.

8.5 Offline Destination Selection

Previously selected places are cached locally in a small bounded list. Offline, the user can reuse a saved destination or tap the map to choose a coordinate. If matching saved route geometry exists, it can be restored. Otherwise the UI truthfully states that internet access is required to generate a new road route. The application does not draw a fake straight line and call it navigation.

8.6 Basemap Tile Caching

The final service worker can cache valid, normally viewed OSM tiles in a bounded cache. The stabilization implementation limits the cache rather than bulk prefetching a city or country. This follows the project's goal of preserving a previously viewed area while respecting that OSM's standard tile service is not an offline-map distribution mechanism [8].

Consequently:

- previously viewed map areas can remain visible offline;
- unseen areas or zoom levels may be unavailable;
- there is no claim of complete offline map coverage.

8.7 Offline SOS and Outbox

IndexedDB provides local-first persistence [10]. Emergency creation is stored locally before depending on successful transmission. On reconnect, queued work is retried. The final lifecycle protects terminal sessions from being intentionally reopened by a replay and avoids making recording success a precondition for SOS persistence.

8.8 Secure Context Requirement

Production phone testing should use HTTPS because geolocation, service workers, push and related browser capabilities depend on secure-context rules in modern browsers [14].

9. Installation and Deployment

9.1 Requirements

The project requires:

- a supported Node.js/npm environment;
- a Supabase project;
- a modern browser; and
- HTTPS for production-like phone/PWA testing.

9.2 Install Dependencies

```
npm install
```

9.3 Environment Variables

The application expects environment variables for the Supabase project and Web Push/VAPID configuration. Documentation must use placeholders only:

```
NEXT_PUBLIC_SUPABASE_URL=<project-url>
NEXT_PUBLIC_SUPABASE_ANON_KEY=<public-anon-key>
SUPABASE_SERVICE_ROLE_KEY=<server-only-service-role-key>
NEXT_PUBLIC_VAPID_PUBLIC_KEY=<public-vapid-key>
VAPID_PRIVATE_KEY=<server-only-vapid-private-key>
```

The service-role key and VAPID private key are secrets. They must never be committed or embedded in client-side code.

9.4 Development

```
npm run dev
```

The production service worker is intentionally not used as a normal development cache mechanism.

9.5 Production Build

```
npm run build
npm start
```

The final stabilization pass completed the normal Next.js Turbopack production build successfully.

9.6 Deployment

The final hackathon build was deployed through a Vercel-connected deployment repository. Production environment variables must be configured in the hosting platform and must not be copied into source control.

9.7 PWA Installation

On supported Android browsers, the application can expose the browser/PWA installation flow. On iPhone/iPad, users can use the browser's Add to Home Screen workflow. The install control is hidden where installation is not applicable, such as an already installed standalone window.

10. Testing and Validation

10.1 Automated Validation

The final stabilization report recorded:

- **101 tests passed, 0 failed** using the explicit Node test command;
- ESLint passed;
- TypeScript `-noEmit` passed;
- `git diff -check` passed;
- normal Next.js Turbopack production build passed; and
- service-worker/check-script syntax checks passed.

A known Node 24 test-directory-resolution issue affects the shorthand `npm test` invocation in the reported environment, while explicitly running the test files executes the suite. The existing module-type warning for `src/lib/offline.ts` is also non-fatal.

10.2 Regression Coverage

The test suite covers areas including:

- offline navigation and distinct cached pages;
- MapLibre worker/module caching;
- safety-overlay restoration;
- saved journey geometry and GPS drift;
- offline map-tap behaviour without fabricated routing;
- service-worker upgrade preservation;
- recording MIME/binary handling and retry safety;
- recorder lifecycle/cleanup;
- SOS persistence despite microphone failure;
- replay-safe emergency lifecycle; and
- push/guardian behaviour from earlier stabilization tests.

10.3 Physical Android Verification

Automated tests do not prove mobile rendering or microphone quality. The final build was therefore also tested on a real Android device. Confirmed behaviour included:

- online GPS, safety dots, search and route generation;
- safety dots remaining visible after going offline;
- the saved route line remaining visible after network loss;
- previously viewed basemap tiles remaining visible in the tested area;
- offline GPS continuing to update; and
- clear UI labelling that offline mode supports saved routes rather than arbitrary new routing.

Cold-reopen behaviour, recording quality and full emergency delivery should still be treated as device/network-dependent checks rather than mathematical guarantees.

11. Demonstration Guide

11.1 Pre-Demo Checklist

Before presenting:

- use the latest stable deployment;
- confirm production environment variables are configured;
- confirm database-backed safety data is available;
- confirm phone location and microphone permissions as appropriate;
- open the app online once so the current service worker/assets can update;
- confirm safety dots and a known route;
- confirm the guardian link/notification path on the second device;
- confirm recordings remain private;
- do not claim guaranteed emergency-service calling or SMS; and
- keep a known-good demonstration journey available.

11.2 Suggested Demo Flow

11.2.1 Step 1: Safety Map

Show the current location and safety circles. Explain that HerGuardian adds a preventive layer before an emergency occurs.

11.2.2 Step 2: Transparent Safety Score

Explain the five factors:

Crime + Reports + Time + Lighting + Crowd

State clearly that the current model is a weighted heuristic, not trained ML.

11.2.3 Step 3: Safer Route Comparison

Select a destination and compare route alternatives. Explain that shortest and safest are separate objectives.

11.2.4 Step 4: Offline Resilience

After loading the route online, disable connectivity and show the retained safety overlay, saved route and previously viewed basemap area. Explain that new arbitrary routing still needs internet.

11.2.5 Step 5: SOS

Activate a safe demonstration SOS. Show local-first persistence, current location, guardian session and the recording status. The final build does not automatically play the earlier siren oscillator.

11.2.6 Step 6: Guardian Alert and Tracking

On a second device, show the guardian notification/session and live/reload-safe location view. Explain that push delivery depends on the browser/OS/network.

11.2.7 Step 7: Voice SOS

Explain the feature accurately: a user can arm Voice SOS before a risky journey and trigger the emergency workflow hands-free while the web app remains active. Do not claim reliable locked-screen/background listening after the app is closed.

11.2.8 Step 8: Community Feedback Loop

Show that anonymous reports can raise nearby report risk, affecting future safety scores and route evaluations.

12. Security and Privacy

12.1 Server-Side Secrets

The Supabase service-role key and VAPID private key are server-only secrets. They must never be exposed through `NEXT_PUBLIC_` variables, client bundles, logs, screenshots or source control.

12.2 Private Emergency Media

The recordings bucket remains private. The system stores private object paths rather than making emergency audio publicly accessible. Binary recording data should not be logged.

12.3 Anonymous Reports

The MVP permits anonymous community submissions, but raw reports are not intended to be publicly enumerable. This reduces unnecessary exposure of potentially sensitive incident information.

12.4 Guardian Session Privacy

The current capability-link model is suitable for a hackathon MVP but weaker than authenticated guardian accounts. Guardian links should be treated as sensitive and should not be posted publicly.

12.5 Push Isolation

Push subscriptions are associated with the relevant guardian session. This avoids intentional global fan-out of one user's SOS to unrelated subscriptions.

12.6 Location Privacy

Precise location is used only where needed for safety, routing and emergency tracking. Production development should minimize unnecessary logging and define clear retention policies before real-world deployment.

13. Known Limitations

The final hackathon prototype has deliberate limitations:

- **Safety data quality.** Synthetic/fallback risk data is not a substitute for validated municipal or police data.
- **Heuristic model.** The safety score is transparent and deterministic, but its weights are not a validated prediction of real-world victimization risk.
- **Public routing dependency.** New road routes require online OSRM access.
- **No arbitrary offline routing.** Offline mode reuses saved route geometry; it does not contain a complete local road-routing engine.
- **Partial offline basemap.** Only previously requested/cached map tiles can be expected offline; unseen coverage may be blank.
- **Saved places are not offline geocoding.** New textual place searches require the online geocoder.
- **Browser lifecycle limits.** Voice detection, recording and background execution can be suspended when the PWA is closed/backgrounded or the device is locked.
- **Voice SOS is armed, not always-on.** It should not be marketed as reliable locked-screen activation.
- **Push is not guaranteed delivery.** It depends on network, browser, OS and notification permission.
- **Calls/SMS require user interaction.** The PWA does not silently place cellular calls or silently send SIM-based SMS messages.
- **Recording limitations.** Abrupt OS termination may lose an unfinished segment; playback/metadata can vary by browser/container implementation.
- **Capability guardian access.** Possession of a valid session link currently grants access instead of a full authenticated guardian relationship.
- **Hackathon prototype.** HerGuardian is not a certified emergency-response service and should not be represented as a replacement for official emergency services.

14. Future Improvements

Potential next steps include:

- replace synthetic/fallback safety factors with validated local datasets;
- empirically calibrate and evaluate the safety model;
- introduce spatial indexing for larger safety datasets;
- add authenticated guardian relationships and revocable invitations;
- build a native mobile application for stronger background/locked-screen emergency capabilities;
- evaluate native emergency voice activation with explicit privacy controls;
- integrate an approved server-side SMS/calling provider where legally and operationally appropriate;
- provide production-grade routing or self-hosted routing infrastructure;
- explore licensed/offline map packages for deliberate offline coverage;
- strengthen media retention, encryption and audit policies;
- build privacy-preserving aggregate authority analytics;
- improve accessibility, localization and multilingual emergency interaction; and
- conduct real-user usability, safety and threat-model testing before production deployment.

15. Conclusion

HerGuardian demonstrates a practical safety architecture built around a single closed loop: predict risk, help prevent exposure, respond during an emergency, and incorporate community feedback into future evaluations.

The final prototype combines a transparent weighted safety model, route safety comparison, real geolocation, anonymous reporting, local-first SOS persistence, guardian tracking, session-specific push alerts, private emergency audio, saved journeys, offline safety overlays, bounded basemap caching and an installable PWA interface.

Its strongest technical principle is graceful degradation. When connectivity is available, the system can search places, generate new routes, synchronize SOS state and deliver online services. When connectivity disappears, it preserves what it can truthfully support: cached safety information, previously viewed map areas, saved destinations, saved route geometry and locally persisted emergency state. It does not pretend to provide arbitrary offline routing or guaranteed emergency communications where the browser platform cannot do so.

The final implementation therefore serves as a hackathon-ready proof of concept and a foundation for a future production system with validated safety data, stronger authentication, native background capabilities and formal safety validation.

Bibliography

- [1] Vercel, *Next.js Documentation*.
<https://nextjs.org/docs>
- [2] Supabase, *Supabase Documentation*.
<https://supabase.com/docs>
- [3] Supabase, *Realtime Documentation*.
<https://supabase.com/docs/guides/realtime>
- [4] Supabase, *Storage Documentation*.
<https://supabase.com/docs/guides/storage>
- [5] MapLibre, *MapLibre GL JS Documentation*.
<https://maplibre.org/maplibre-gl-js/docs/>
- [6] Project OSRM, *OSRM API Documentation*.
<https://project-osrm.org/docs/v5.24.0/api/>
- [7] OpenStreetMap Foundation, *Nominatim Usage Policy*.
<https://operations.osmfoundation.org/policies/nominatim/>
- [8] OpenStreetMap Foundation, *Tile Usage Policy*.
<https://operations.osmfoundation.org/policies/tiles/>
- [9] MDN Web Docs, *Service Worker API*.
https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
- [10] MDN Web Docs, *IndexedDB API*.
https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
- [11] World Wide Web Consortium, *MediaStream Recording*.
<https://www.w3.org/TR/mediastream-recording/>
- [12] World Wide Web Consortium, *Push API*.
<https://www.w3.org/TR/push-api/>
- [13] World Wide Web Consortium, *Geolocation API*.
<https://www.w3.org/TR/geolocation/>
- [14] MDN Web Docs, *Secure Contexts*.
https://developer.mozilla.org/en-US/docs/Web/Security/Secure_Contexts
- [15] MDN Web Docs, *Web Share API*.
https://developer.mozilla.org/en-US/docs/Web/API/Web_Share_API

HerGuardian – Team Friday
Final Hackathon Technical Documentation – October 2026